

Avoiding Locks: Read-Copy-Update

Concurrent reads with carefully published updates

Definition

Read-Copy-Update, or **RCU**, is a synchronization technique that lets readers access a shared data structure without taking locks, while writers publish new versions carefully and reclaim old versions only after readers are done.

1. Why Avoid Locks?

Main Idea

The fastest lock is no lock at all. If readers can safely run without locking, read-heavy systems can become much faster.

Not Always Possible

In the general case, concurrent reading and writing without locks is unsafe. A reader may observe a temporary, inconsistent state created while a writer is changing the data.

Unsafe Array Example

Suppose process A is sorting an array while process B is computing the average. Because A moves values around, B may count some values twice and miss others entirely. The computed average may be wrong.

2. The RCU Trick

Key Rule

Each reader should see either the old version of the data structure or the new version, but not a broken mixture of both.

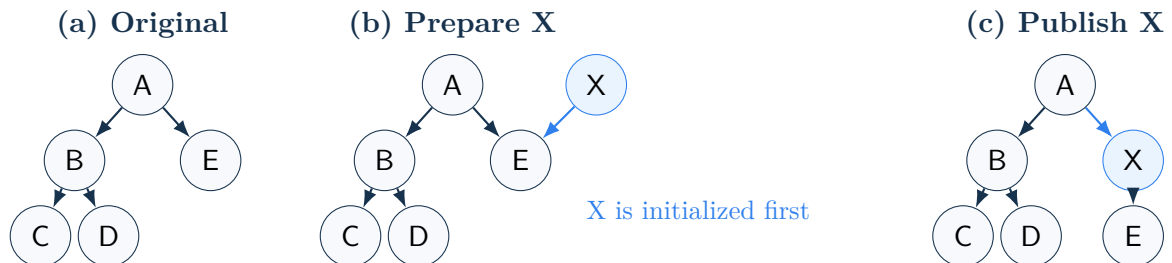
RCU works well for pointer-based data structures such as linked lists and trees. A writer prepares a new or changed piece of the structure privately, then makes it visible with a small atomic pointer update.

Step	Meaning
Read	Readers traverse the current data structure without taking locks.
Copy / prepare	A writer builds or modifies a new version off to the side.
Update	The writer publishes the new version with an atomic pointer change.
Reclaim later	Old memory is freed only after old readers can no longer reference it.

3. Adding a Node Safely

Insertion Idea

Before a new node becomes reachable from the shared tree, the writer completely initializes it. After that, a single atomic pointer update makes the node visible.



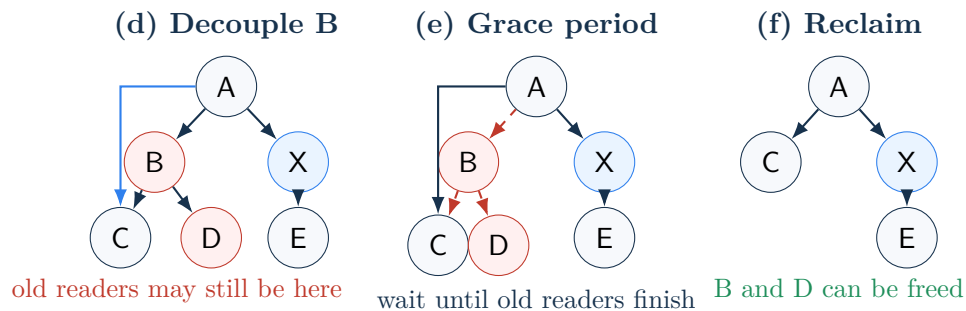
Why Insertion Is Safe

Readers already at node E continue through the old path. Readers that start from A after the atomic update see X fully initialized. No reader sees a half-built node.

4. Removing Nodes Safely

Removal Idea

RCU removal has two separate phases: first remove the old nodes from the shared structure, then reclaim their memory later.



Do Not Free Immediately

After B and D are removed from the public tree, some readers may still hold references to them. Freeing them immediately could leave those readers with dangling pointers.

5. Removal vs. Reclamation

Decoupling

RCU decouples **removal** from **reclamation**. Removal makes old nodes unreachable by new readers. Reclamation frees their memory after old readers are gone.

Phase	Meaning
Removal	Change pointers so new readers no longer reach the old nodes.
Grace period	Wait long enough that every old reader has finished its read-side critical section.
Reclamation	Free the removed nodes safely.

Core Insight

The writer does not need to stop readers from using old nodes. It only needs to avoid freeing those nodes until the readers are finished.

6. Read-Side Critical Sections

Read-Side Critical Section

A **read-side critical section** is the part of a reader that accesses the RCU-protected data structure.

Important Restriction

In the simplified model, code inside an RCU read-side critical section must not block or sleep.

```
rcu_read_lock()

    traverse shared tree
    read nodes and follow pointers

rcu_read_unlock()
```

Why Blocking Is Forbidden

If a reader could sleep while holding a pointer into the old structure, the writer might have to wait an unbounded amount of time before freeing old nodes.

7. Grace Period

Grace Period

A **grace period** is a time interval during which every thread is known to have been outside its RCU read-side critical section at least once.

Safe Reclamation Rule

After a full grace period has passed, readers that could have seen the old version are finished. The removed nodes can then be reclaimed.

Context Switch Criterion

If read-side critical sections cannot block or sleep, one simple way to detect a grace period is to wait until all threads have executed a context switch.

8. Writer Sketch

```
new_node := allocate node
initialize all fields in new_node
initialize all child pointers in new_node

atomically publish pointer to new_node
```

```
old_nodes := remove nodes from public structure

wait for grace period

free old_nodes
```

Atomic Publish

The publishing step must be atomic from the readers' point of view. A reader should never observe a pointer to a partially initialized node.

9. When RCU Works Well

Good Fit	Reason
Read-heavy data	Readers avoid lock overhead.
Pointer-based structures	Writers can publish new links atomically.
Short read-side sections	Grace periods finish quickly.
Old versions can coexist briefly	Readers can safely finish on the version they already saw.

10. Limitations

Not a General Replacement for Locks

RCU does not make every shared update safe. It is most useful when readers can tolerate seeing either an old version or a new version.

Memory Reclamation Is Hard

The writer must know when old readers are gone. Reclaiming too early can cause use-after-free errors.

Reader View

RCU gives readers a consistent traversal, not necessarily the newest possible data. A reader may legally see the old version while another reader sees the new version.

11. RCU vs. Locks

Feature	Locks	RCU
Reader cost	Readers acquire and release a lock.	Readers usually avoid normal lock acquisition.
Writer behavior	Writer often excludes readers.	Writer publishes a new version while old readers continue.
Reader view	Usually sees the protected current state.	Sees either an old consistent version or a new consistent version.
Memory freeing	Freeing can happen after the lock-protected update.	Freeing must wait for a grace period.
Best use	General mutual exclusion.	Read-heavy pointer-based data structures.

12. Big Picture

Main Conclusion

RCU avoids locking readers by allowing old and new versions of a data structure to coexist temporarily. Writers publish updates atomically and reclaim removed data only after a grace period proves that old readers are gone.

Exam Shortcut

Remember: RCU means readers run without normal locks, writers copy or prepare updates, publish with an atomic pointer change, wait for a grace period, and only then reclaim old memory.

13. Quick Review

Key Points

- Avoiding locks can improve performance, especially for read-heavy data.
- Lock-free concurrent access is not safe in the general case.
- RCU makes sure readers see either the old version or the new version.
- A new node is fully initialized before it is made visible.
- Publishing is done with an atomic pointer update.
- Removing nodes does not mean freeing them immediately.
- RCU separates removal from memory reclamation.
- Removed nodes are freed only after a grace period.
- A grace period proves that old readers have left their read-side critical sections.
- Read-side critical sections should not block or sleep in the simplified model.
- RCU is useful for read-heavy pointer-based structures, but it is not a universal replacement for locks.